دانشگاه لرستان

پروژه پایانی برنامه نویسی پیشرفته

مهلت تحویل: ۱۴۰۵/۰۴/۲۷

دکتر آرمین رشنو

عضو هیئت علمی دانشگاه لرستان و دانشیار هوش مصنوعی

WebPage

عنوان پروژه

طراحی و پیاده‌سازی سیستم ساده انتخاب واحد دانشگاهی

اهداف پروژه:

- ✓ **اشیا و برنامه‌نویسی شی‌گرا**
- ✓ **ارث‌بری**
- ✓ **مدیریت استثنا**
- ✓ **مفاهیم مقدماتی** HTTP
- ✓ **برنامه‌نویسی** API **با** FastAPI
- ✓ **اعتبارسنجی مقادیر مجاز برای داده ها (Data Validation)**
- ✓ **استفاده از** redsponse model**ها برای کنترل خروجی ها به سمت کاربر**
- ✓ **آشنایی مقدماتی با** path parameters **و** query parameters **و** body **در** FastAPI
- ✓ **آشنایی با مفهوم** routers **در** FastAPI
- ✓ **آشنایی با ابزارهای تست** API **ها شامل** Postman **و** Insomnia
- ✓ **استفاده از ابزار** Github
- ✓ **بارگذاری بر روی سرور و ارائه سرویس بصورت عمومی(Valid IP)**

بخش اول

طراحی کلاس ها: پکیج models

در این پکیج همه کلاس های لازم ایجاد می شوند. این کلاس ها در پکیج های دیگر مورد استفاده قرار میگیرند.

۱–کلاس پایه Person :

ویژگی ها: کد ملی **ID** و نام(**first_name**) و نام خانوادگی(**last_name**) است.

توابع: تابع **__init__** برای مقدار دهی اولیه به اشیایی که از این تابع ساخته می شوند. که اجازه ایجاد اشیا بدون فیلد را نمیدهد. تابع **get_full_name** است که نام و نام خانوادگی را به هم میچسباند و نام کامل را برمیگرداند. تابع **to_dict** برای تبدیل ویژگی ها به یک ساختار **Json** است که بتوان آن را در یک دیکشنری ذخیره کرد.

۲–کلاس Professor : از کلاس پایه **Person** ارث بری می کند.

ویژگی ها: ۳ ویژگی کلاس پایه را به ارث می برد دارای ۳ ویژگی اضافه شماره استادی (**personnel_code**) و دانشکده(**department**) و لیست دروس ارائه شده (**courses**) می باشد.

توابع: تابع **__init__** برای مقدار دهی اولیه به اشیایی که از این تابع ساخته می شوند که بخشی از آن را از کلاس پایه به ارث می برد. تابع **assign_course** برای تخصیص یک درس به استاد است. تابع **get_courses** برای برگرداندن همه درس هایی که یک استاد ارائه می کند. تابع **to_dict** برای تبدیل ویژگی ها به یک ساختار **Json** است که بتوان آن را در یک دیکشنری ذخیره کرد.

۳–کلاس Student : از کلاس پایه **Person** ارث بری می کند.

ویژگی ها: ۳ ویژگی کلاس پایه را به ارث می برد دارای ۳ ویژگی اضافه شماره دانشجویی (**student_number**) و رشته تحصیلی (**major**) و لیست دروس انتخاب شده (**selected_courses**) می باشد.

توابع: تابع **__init__** برای مقدار دهی اولیه به اشیایی که از این تابع ساخته می شوند که بخشی از آن را از کلاس پایه به ارث می برد. تابع **select_course** برای انتخاب درس توسط دانشجو می باشد. تابع **drop_cours**e برای حذف یک درس می باشد. تابع **get_courses** برای برگرداندن همه درس هایی که دانشجو اخذ کرده است. تابع **to_dict** برای تبدیل ویژگی ها به یک ساختار **Json** است که بتوان آن را در یک دیکشنری ذخیره کرد.

۴–کلاس Course :

ویژگی ها: شماره درس، عنوان، کد، تعداد واحد، ظرفیت، استاد، دانشجویان **توابع:** تابع **__init__** برای مقدار دهی اولیه به اشیایی که از این تابع ساخته می شوند، تابع پر بودن درس، تابع اضافه کردن دانشجو به درس، تابع حذف دانشجو از درس، تابع تخصیص درس به استاد، تابع **to_dict** برای تبدیل ویژگی ها به یک ساختار **Json** است که بتوان آن را در یک دیکشنری ذخیره کرد.

هر ۴ کلاس در یک پکیج با نام models قرار دارند. یعنی هر کلاس به همراه ویژگی ها و توابع خودش در یک ماژول با پسوند py. قرار دارد و یک فایل خالی با نام __init__.py هم داخل این پوشه قرار دارد که آن را به پکیج تبدیل می کند تا قابل فراخوانی توسط پکیج ها یا ماژول های دیگر باشد.

بخش دوم

طراحیِ شما ها: پکیج schemas

در این پکیج کلاس هایی وجود دارد که از مدل اعتبارسنجی Pydantic استفاده میکنند و قالب استاندارد برای ورودی ها را مشخص میکنند. یعنی مقادیر درست را برای فیلدهای هر کلاس اعتبارسنجی میکنند و در صورتی که مقادیر معتبر نباشند اجازه ثبت نمیدهند.

۱–ماژول student_schema

کلاس **StudentUpdate**

این شِما ورودی های لازم هنگام ایجاد دانشجو را بررسی و اعتبارسنجی میکند.

first_name: از نوع رشته حداقل طول ۲ و حداکثر طول ۵۰ می باشد

last_name: str = از نوع رشته حداقل طول ۲ و حداکثر طول ۵۰ می باشد

student_number: از نوع رشته حداقل طول ۳ و حداکثر طول ۲۰ می باشد

major: از نوع رشته حداقل طول ۲ و حداکثر طول ۸۰ می باشد

کلاس **StudentUpdate**

همان شِمای **StudentCreate** است فقط مقدار پیش فرض برای فیلدها None است.

۲–ماژول professor_schema

کلاس **ProfessorCreate**

این شِما ورودی های لازم هنگام ایجاد استاد را بررسی و اعتبارسنجی میکند.

first_name: از نوع رشته حداقل طول ۲ و حداکثر طول ۵۰ می باشد

last_name: str = از نوع رشته حداقل طول ۲ و حداکثر طول ۵۰ می باشد

personnel_code: از نوع رشته حداقل طول ۳ و حداکثر طول ۲۰ می باشد

department: از نوع رشته حداقل طول ۲ و حداکثر طول ۸۰ می باشد

کلاس **ProfessorUpdate**

همان شِمای **ProfessorCreate** است فقط مقدار پیش فرض برای فیلدها None است.

۳–ماژول course_schema

کلاس **CourseCreate**

این شِما ورودی های لازم هنگام ایجاد درس را بررسی و اعتبارسنجی میکند.

major: از نوع رشته حداقل طول ۲ و حداکثر طول ۸۰ می باشد

title: از نوع رشته حداقل طول ۲ و حداکثر طول ۱۰۰ می باشد

code: از نوع رشته حداقل طول۲ و حداکثر طول ۲۰ می باشد

unit: از نوع عدی حداقل مقدار ۱ و حداکثر مقدار ۵ می باشد.

capacity: از نوع عدی حداقل مقدار ۱ و حداکثر مقدار ۲۰۰ می باشد

کلاس **CourseUpdate**

همان شِمای **CourseCreate** است فقط مقدار پیش فرض برای فیلدها None است.

بخش سوم

ذخیره داده ها: پکیج data

در این پکیج داده های مربوط به ۳ موجودیت دانشجو، استاد و درس در فایل های **json** در مسیر جاری پروژه ذخیره می شوند. ذخیره بصورت دائمی است یعنی با خروج از برنامه، داده ه از دست نمی روند.

۱–ماژول storage

ابتدا ۳ فایل json با محتوی خالی برای هر موجودیت ایجاد شود. سپس ۳ شمارنده با مقدار اولیه ۱ برای هر کدام ایجاد شود که هر بار با اضافه شدن یک رکورد(شی) از آن موجودیت ۱ واحد به شمارنده اضافه می شود.

تابع _read_json

آدرس یک مسیر را از ورودی میگیرد و همه داده های ذخیره شده در آن مسیر را برمیگرداند.

تابع _write_json

یک داده و آدرس یک مسیر را از ورودی میگیرد و داده را در آن مسیر ذخیره میکند.

تابع save_all

محتوی هر ۳ موجودیت دانشجو، استاد و درس را ذخیره میکند. این تابع از تابع **write_json_** برای نوشتن همه رکوردها استفاده میکند.

تابع load_all

محتوی هر ۳ موجودیت دانشجو، استاد و درس را بارگذاری میکند.

تابع reset_storage

محتوی همه داده های ذخیره شده در پروژه را پاک میکند.

بخش چهارم

طراحی سرویس ها: پکیج services

در این پکیج سرویس های مورد نیاز برای هر کلاس پیاده سازی می شود. منظور از سرویس عملیات است. به عنوان مثال عملیات ایجاد دانشجو، حذف دانشجو، ایجاد درس، تخصیص درس به دانشجو و ... .

۱–ماژول student_services

تابع create_student

برای ایجاد یک دانشجوی جدید می باشد. اگر دانشجو از قبل ایجاد شده باشد اجازه ایجاد شدن ندهد. یک شی از کلاس Student ایجاد می کند و شی آن را در قالب json ذخیره میکند. ساختار ذخیره در بخش های بعد تشریح خواهد شد. بعد از ایجاد و ذخیره، شی آن را به ما بر میگرداند.

تابع get_all_students

لیست همه دانشجویان ثبت شده را برمیگرداند. همه دانشجویان در یک لیست از اشیا ذخیره می شوند. که هر شی مربوط به یک دانشجو می باشد.

تابع get_student_by_id

شماره دانشجو را میگیرد و مشخصات آن دانشجو را برمرگرداند. اگر دانشجو وجود نداشته باشد با استثنا آن را مدیریت میکند و پیغام مناسب را صادر میکند.

تابع update_student

برای بروزرسانی مشخصات یک دانشجو استفاده می شود. شماره دانشجو و یک شی از کلاس **StudentUpdate** در ماژول schema میگیرد و برای مقادیر ارسالی که غیر None باشند مقدار آن فیلد را بروزرسانی می کند. عدم وجود دانشجو و دانشجوی تکراری بودن را با استثنا مدیریت میکند.

تابع delete_student

برای حذف یک دانشجو استفاده می شود. شماره دانشجو را میگیرید و آن را حذف می کند. اگر دانشجو وجود نداشت با استثنا مدیریت میکند.

۲–ماژول professor_services

توابع آن دقیقا مانند student می باشند.

۳–ماژول course_services

توابع آن دقیقا مانند student می باشند.

۴–ماژول selection_services

تابع select_course_for_student

یک شماره دانشجو و یک شماره درس را میگیرد و آن درس را برای دانشجو انتخاب میکند.

تابع drop_course_for_student

یک شماره دانشجو و یک شماره درس را میگیرد و آن درس را از لیست دروس آن دانشجو حذف میکند.

تابع get_student_courses

یک شماره دانشجو میگیرد و لیست همه دروس انتخاب شده توسط آن دانشجو را برمیگرداند.

تابع assign_professor_to_course

یک شماره استاد و یک شماره درس را میگیرد و آن درس را به استاد تخصیص میدهد.

بخش پنجم

مدیریت استثناها: پکیج exceptions

در این پکیج سرویس های مورد نیاز برای هر کلاس پیاده سازی می شود. منظور از سرویس عملیات است. به عنوان مثال عملیات ایجاد دانشجو، حذف دانشجو، ایجاد درس، تخصیص درس به دانشجو و ... .

۱–ماژول custom_exceptions

تابع create_student

دارای کلاس پایه CourseSelectionException است که از کلاس پایه Exception ارث بری می کند. همه کلاس های تعریف شده در این پروژه برای مدیریت استثنا از CourseSelectionException ارث بری میکنند و این کلاس ها بدون هیچ شرطی پیغام خطا را pass میکند تا نمایش داده شود.

بخش ششم

مسیردهی درخواست ها: پکیج routers

در FastAPI روتر برای مسیردهی به درخواست های کاربران استفاده می شود. وقتی درخواست ها به هسته اصلی پروژه میرسند از طریق ۳ انشعاب(شاخه) دانشجو، استاد و درس درخواست ها را هدایت میکنیم تا هر درخواستی در شاخه مربوط به خودش پاسخ داده شود.

۱–ماژول students

همه API های(endpoints) مربوط به دانشجو در این قسمت پیاده سازی می شوند. دسترسی به این مسیر برای کاربران از پیشوند students استفاده میکنند. یعنی برای دسترسی به APIهای این قسمت از آدرس زیر استفاده میکنند:

/students/آدرس سرور ما/

API post

این API یک شی از نوع StudentCreate در ماژول schemas را به عنوان بدنه میگیرد. و با استفاده از تابع create_student در ماژول services دانشجو را ایجاد می کند و ذخیره میکند. سپس شی دانشجو را با استفاده از تابع to_dict به ساختار json تبدیل میکند و برای کاربر برمیگرداند.

API get

این API لیست همه دانشجویان ثبت شده را در قالب json برای کاربر برمیگرداند. هر دانشجو در قالب یک شی ذخیره شده است. برای تبدیل شی به json از تابع to_dict استفاده شود.

API get("/{student_id}")

این API شماره دانشجویی را بصورت path parameter از کاربر میگیرد و با استفاده از تابع get_student_by_id در پکیج services مشخصات آن دانشجو را در قالب json برای کاربر برمیگرداند. هر دانشجو در قالب یک شی ذخیره شده است. برای تبدیل شی به json از تابع to_dict استفاده شود.

API put("/{student_id}")

این API شماره دانشجویی را بصورت path parameter از کاربر میگیرد و با استفاده از تابع update_student در پکیج services مشخصات آن دانشجو را بروزرسانی میکند و مشخصات بروزرسانی شده را در قالب json برای کاربر برمیگرداند. هر دانشجو در قالب یک شی ذخیره شده است. برای تبدیل شی به json از تابع to_dict استفاده شود.

API delete("/{student_id})

این API شماره دانشجویی را بصورت path parameter از کاربر میگیرد و آن دانشجو را حذف میکند و این پیغام را برای کاربر برمیگرداند: . دانشجو با موفقیت حذف شد.

API get("/{student_id}/courses")

این API شماره دانشجویی را بصورت path parameter از کاربر میگیرد و لیست دروس انتخاب شده توسط آن دانشجو را در قالب json برای کاربر برمیگرداند.برای تبدیل شی به json از تابع to_dict استفاده شود.

API انتخاب درس برای دانشجو Post

API حذف درس برای دانشجوdelete

.

۲–ماژول professors

APIهای آن شبیه students است.

۲–ماژول courses

APIهای آن شبیه students است.

بخش هفتم

هسته اصلی پروژه: ماژول main

هسته اصلی پروژه در ماژول main.py قرار دارد و همه درخواست ها اولیه به این قسمت ارجاع داده می شوند.

ابتدا یک شیapp از fastapi ساخته شود و به همه روترهای ایجاد شده در پکیج routers وصل شود.

سپس مسیر ریشه یک پیعام خوشامدگویی برای کاربران نمایش داده شود.

زمان شروع سرور برای اجرای APIها همه داده های ذخیره شده بارگذاری شوند.

زمان خاموش شدن سرور با روش های نرمال، همه داده ها ذخیره شوند.

خلاصه همه داده ها در یک API نمایش داده شود: تعئتئ همه دانشجویان ثبت نام شده، تعداد همه اساتید، تعداد دروس و ...

لیست همه داده ها(دانشجو، استاد، درس) در یکapi نمایش داده شود.

همه خطاهای مدیریت شده در همه بخش های قبلی به اینجا منتقل شده و برای کاربر نمایش داده شوند. یعنی در بستر API استفاده شوند.

نیازمندی های پروژه: requirements

نیازمندی های پروژه fastapi و uvicorn و pydantic می باشند که در فایل reqqqqqquirements.txt در مسیر جاری پروژه قرار میگیرد.

بخش هشتم

داکرایز کردن پروژه: Docker

پروژه را داکرایز کرده تا image ها و container ها بر روی هر سیستم مقصدی قابل اجرا باشند.

بخش نهم

خرید دامنه: Domain

در سایت ایرنیک(nic.ir) یک دامنه با نام و فامیلی خودتان ثبت کنید. (به عنوان مثال arminrashno.ir). اگر قبلا گرفته با کم و زیاد کردن کاراکترها یک مورد آزاد را پیدا کنید. دامنه را به بستر cloudflare منتقل کنید تا ترافیک سرویس شما در این بستر مدیریت شود. سپس از cloudflare ترافیک را به سرور خودتان هدایت کنید.

بخش دهم

راه اندازی بر روی سرور

یک سرور لینوکسی تهیه کنید(میتوانید از نسخه های رایگان استفاده کنید) و پروژه را بر روی سرور راه اندازی کنید. به گونه ای که همه API ها ی پیاده سازی شده شما روی دامنه شما در بستر اینترنت قابل دسترس باشند.

منابع

منبع اصلی	فیلم های آموزشی ارسالی برای دانشجویان
منبع خلاصه	https://www.mongard.ir/courses/fastapi/
منبع کامل	https://fastapi.tiangolo.com/

موفق باشید.

رشنو.